1. OpenClaw Deployment

========== [Note] ==========

This section is the complete installation and deployment tutorial for OpenClaw. It involves many steps, so if you want to use OpenClaw quickly, please

follow the tutorial in the first section of this chapter: [1. Directly Use the Pre-configured Environment (Recommended)]

==========

1. OpenClaw Deployment

- 1.1 OpenClaw Installation
  - 1.1.1 Installing OpenClaw
  - 1.1.2 Installation using QuickStart
  - 1.1.3 Opening the OpenClaw Control UI on a PC (Windows)
- 1.2 Configuring the Model (Must Read)
  - 1.2.1 Why a model must be configured
  - 1.2.2 How to configure a model
  - 1. OpenRouter (an OpenClaw built-in provider)
  - 2. Bailian (an example of a third-party provider API)

This chapter covers two parts:

- OpenClaw installation (Jetson Nano Docker scenario)
- Model configuration (OpenRouter / Bailian / Ollama)

1.1 OpenClaw Installation

OpenClaw official website: https://openclaw.ai/

Open source project repository: https://github.com/openclaw/openclaw

1.1.1 Installing OpenClaw

OpenClaw has already been configured in the official Yabo Intelligent image. This section is provided only as a reference in case you want to install it yourself.

> The system version of the Jetson Nano B01 is relatively old and does not support versions above Node 22, so installation via Docker is mandatory.

1.1.2 Installation using QuickStart

We use the installation script officially provided by OpenClaw

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

```
root@jetson-yahboom:/# curl -fsSL https://openclaw.ai/install.sh | bash

  🦞 OpenClaw Installer
  Your messages, your servers, Meta's tears.

✔ Detected: linux

Install plan
OS: linux
Install method: npm
Requested version: latest

[1/3] Preparing environment
· Node.js not found, installing it now
· Installing Node.js via NodeSource
· Installing Linux build tools (make/g++/cmake/python3)
✔ Build tools installed
✔ Node.js v22 installed
· Active Node.js: v22.22.1 (/usr/bin/node)
· Active npm: 10.9.4 (/usr/bin/npm)

[2/3] Installing OpenClaw
✔ Git already installed
· Installing OpenClaw v2026.3.7
✔ OpenClaw npm package installed
✔ OpenClaw installed

[3/3] Finalizing setup

🦞 OpenClaw installed successfully (2026.3.7)!
Settled in. Time to automate your life whether you're ready or not.

· Starting setup


🦞 OpenClaw 2026.3.7 (42a1394) — I run on caffeine, JSON5, and the audacity of "it worked on my machine."
```

During this process, the installation script automatically detects the missing environment and helps us install everything with one click

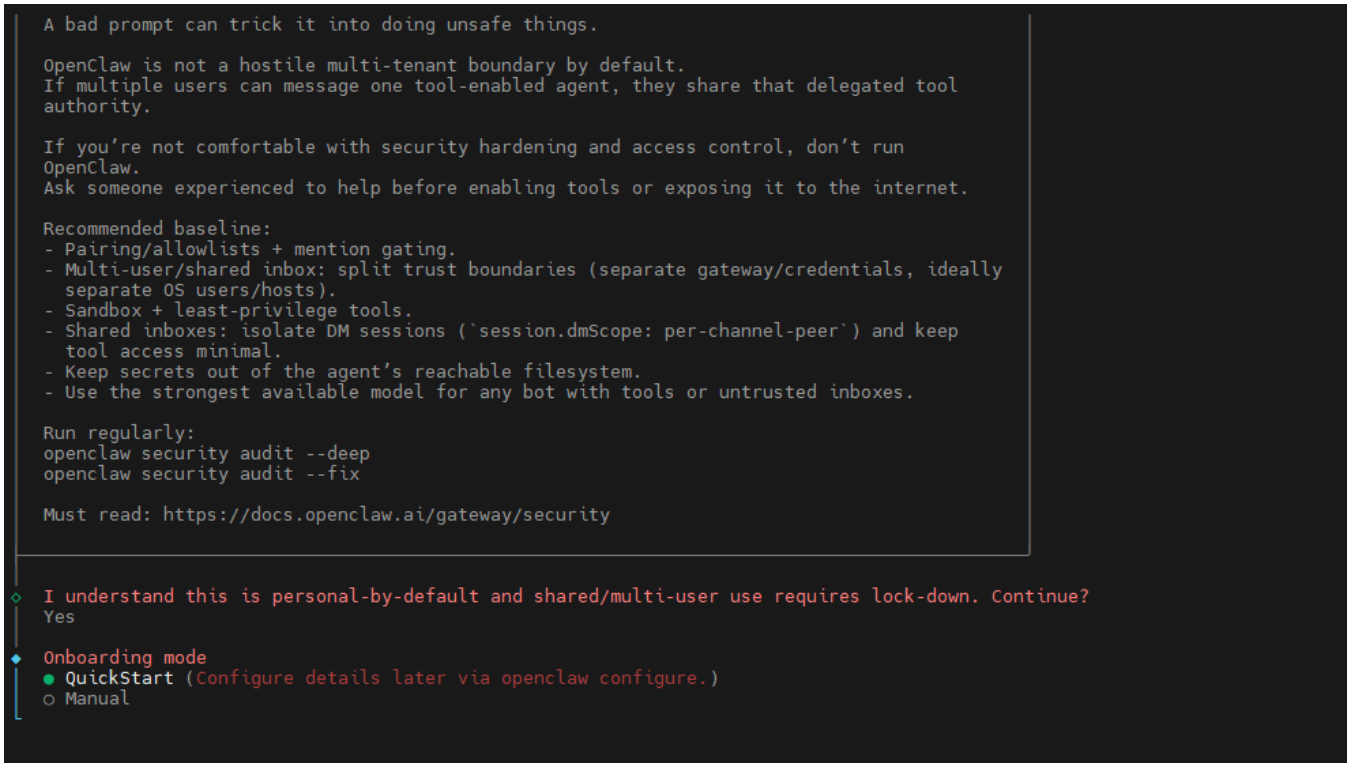
> During the installation, use the Up/Down/Left/Right arrow keys to select an option, the Spacebar to select/deselect, and the Enter key to confirm and configure

```
🦞 OpenClaw 2026.3.7 (42a1394) — I run on caffeine, JSON5, and the audacity of "it worked on my machine."

 ▄▀▀▀▄ ▄▀▀▀▀ ▄▀▀▀▄ ▄▀▀▀▀ ▄▀▀▀▄ ▄▀▀▀▄ ▄▀▀▀▄ ▄▀▀▀▄ ▄▀▀▀▄ ▄▀▀▀▄ ▄▀▀▀▄
 ▀▄▄▄▀ ▀▄▄▄▄ ▀▄▄▄▀ ▀▄▄▄▄ ▀▄▄▄▀ ▀▄▄▄▀ ▀▄▄▄▀ ▀▄▄▄▀ ▀▄▄▄▀ ▀▄▄▄▀ ▀▄▄▄▀

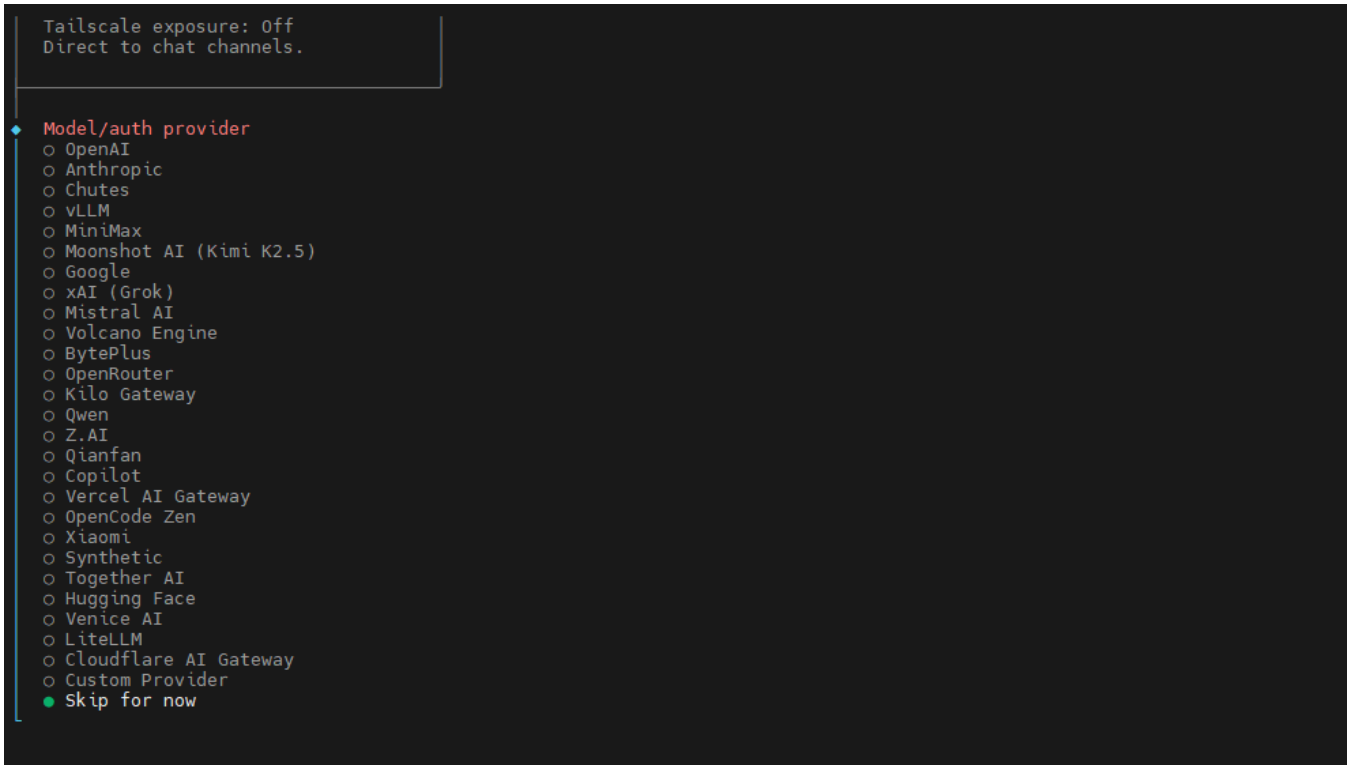
                🦞 OPENCLAW 🦞

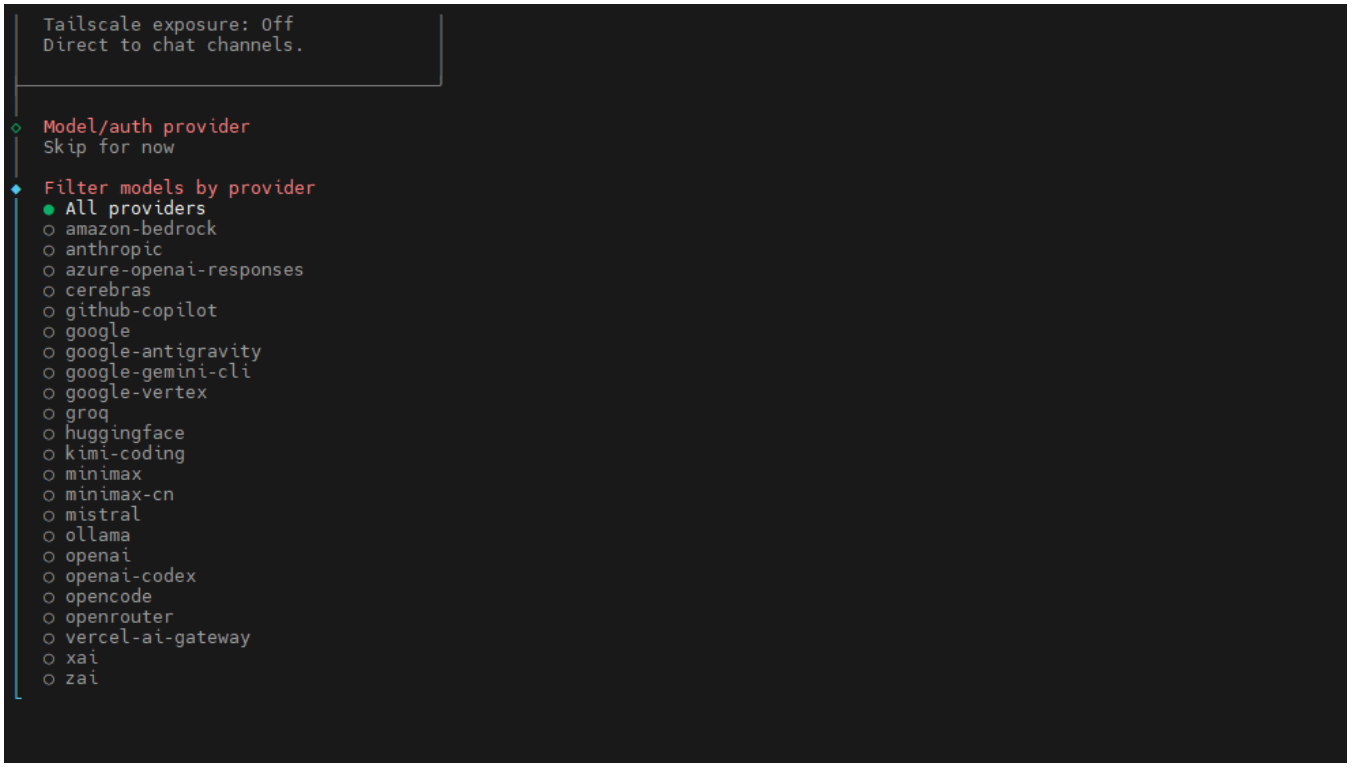
┌  OpenClaw onboarding
│
◇  Security ──────────────────────────────────────────────────────────────────╮
│                                                                            │
│  Security warning — please read.                                           │
│                                                                            │
│  OpenClaw is a hobby project and still in beta. Expect sharp edges.        │
│  By default, OpenClaw is a personal agent: one trusted operator boundary.  │
│  This bot can read files and run actions if tools are enabled.             │
│  A bad prompt can trick it into doing unsafe things.                       │
│                                                                            │
│  OpenClaw is not a hostile multi-tenant boundary by default.               │
│  If multiple users can message one tool-enabled agent, they share that delegated tool
│  authority.                                                                │
│                                                                            │
│  If you're not comfortable with security hardening and access control, don't run
│  OpenClaw.                                                                 │
│  Ask someone experienced to help before enabling tools or exposing it to the internet.
│                                                                            │
│  Recommended baseline:                                                     │
│  - Pairing/allowlists + mention gating.                                    │
│  - Multi-user/shared inbox: split trust boundaries (separate gateway/credentials, ideally
│    separate OS users/hosts).                                               │
│  - Sandbox + least-privilege tools.                                        │
│  - Shared inboxes: isolate DM sessions (`session.dmScope: per-channel-peer`) and keep
```



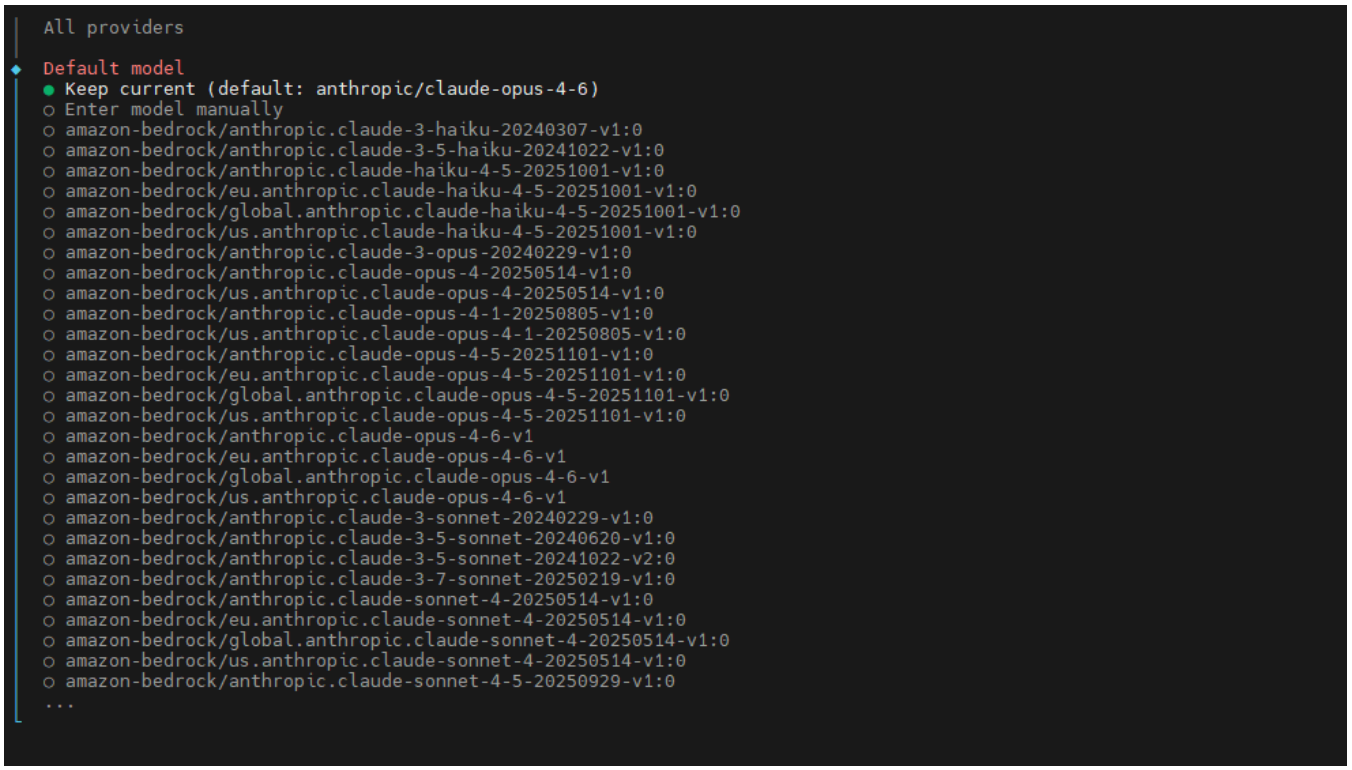
> Here we use the QuickStart option to complete the OpenClaw installation quickly; the detailed configuration can be deferred to later

If you have not applied for any API Key, just choose Skip for all of them here, and configure them later

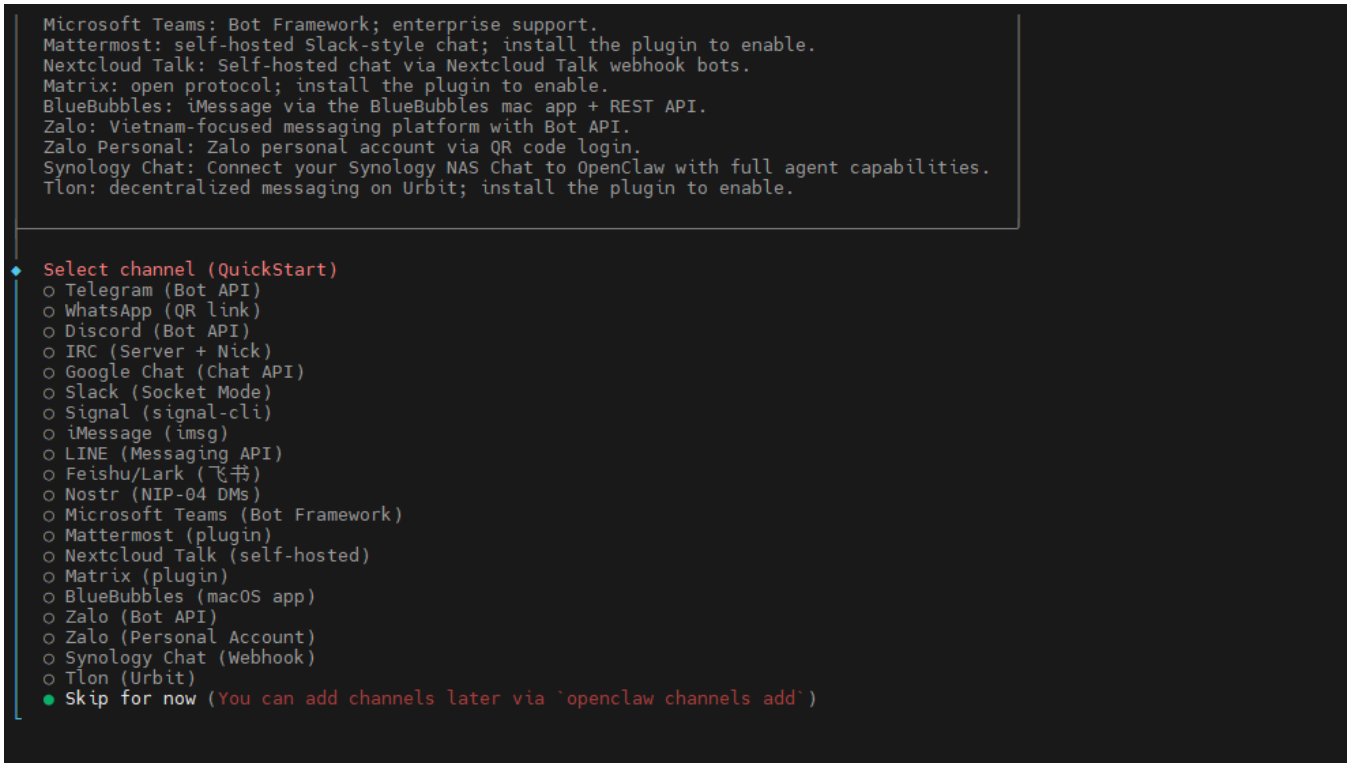




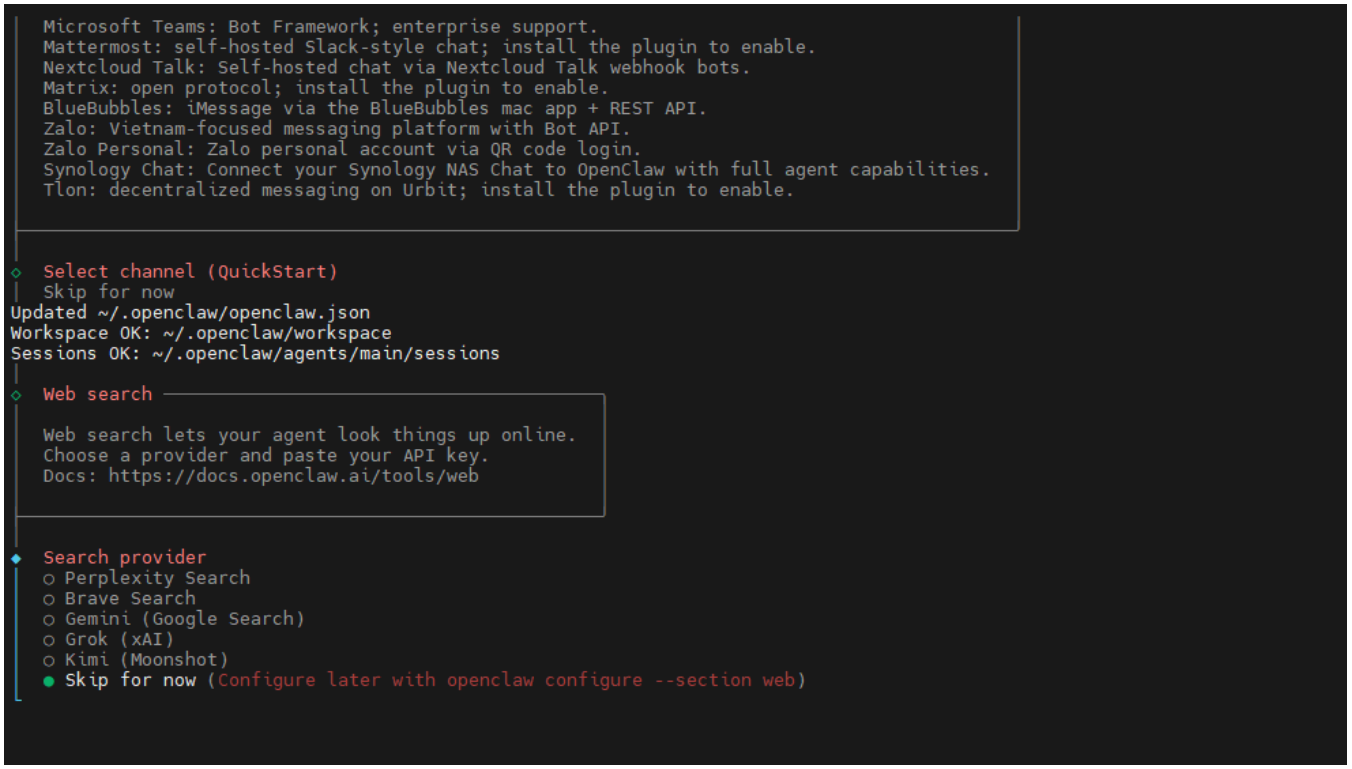
For the model, we choose to keep the default (to be configured later)



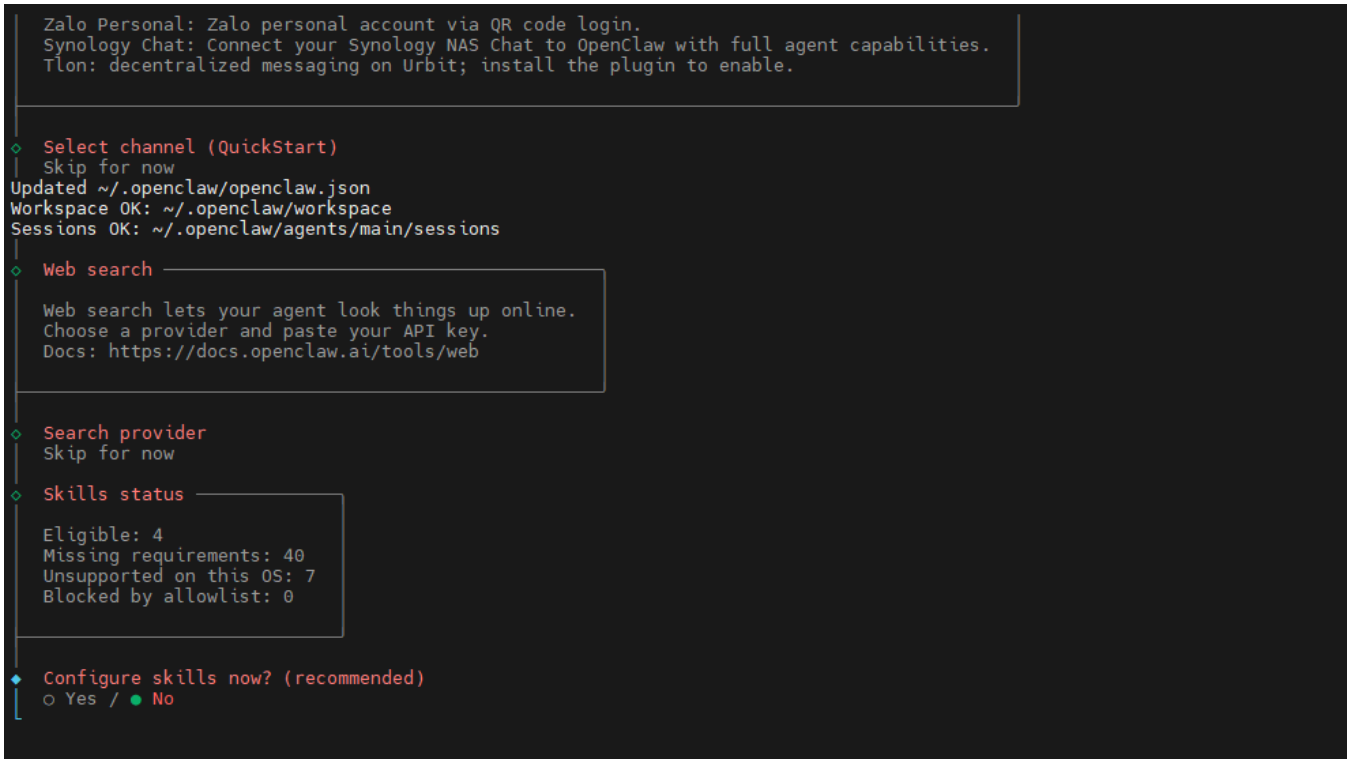
The Channel option is mainly used for communication with third-party APPs, so we skip it for now as well



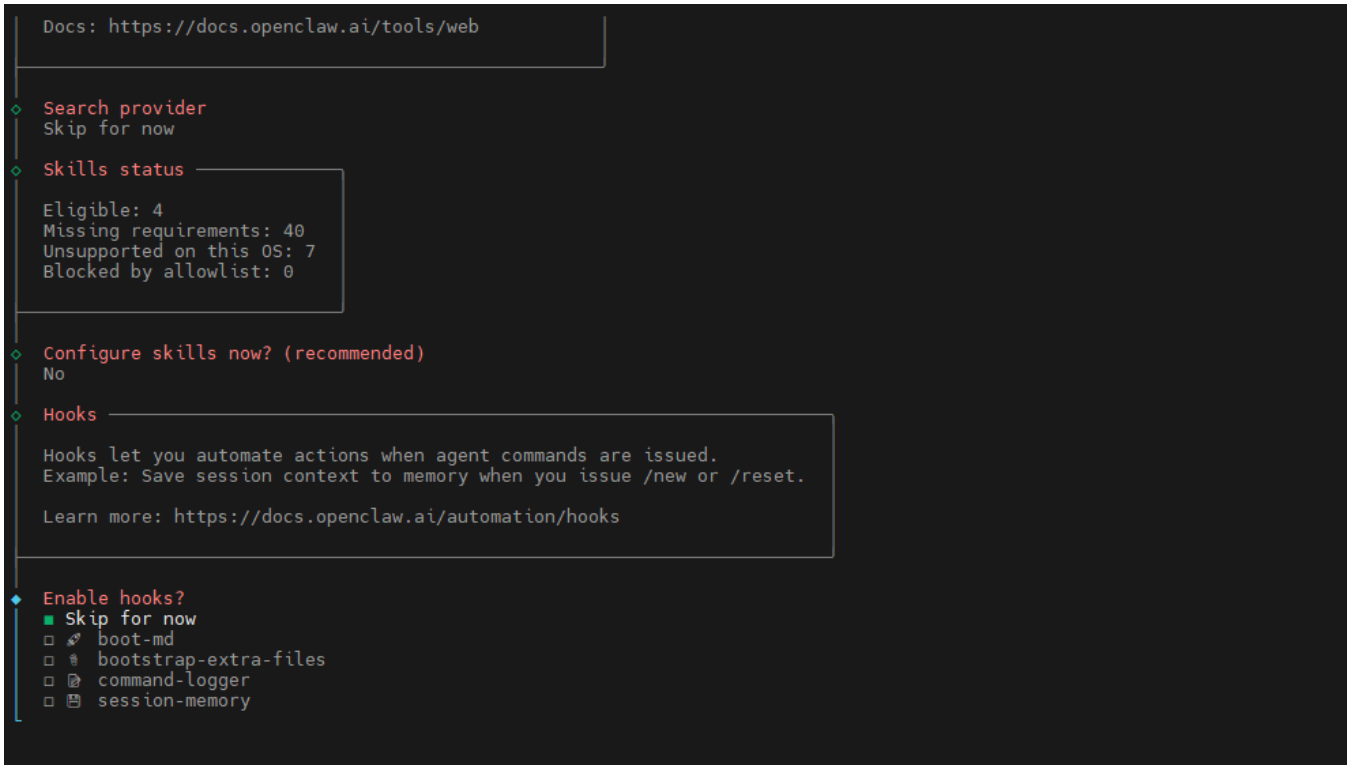
Search provider is the API provider of the search service; we choose to skip it



For the "Skills" configuration, we choose not to configure them for now (select no)

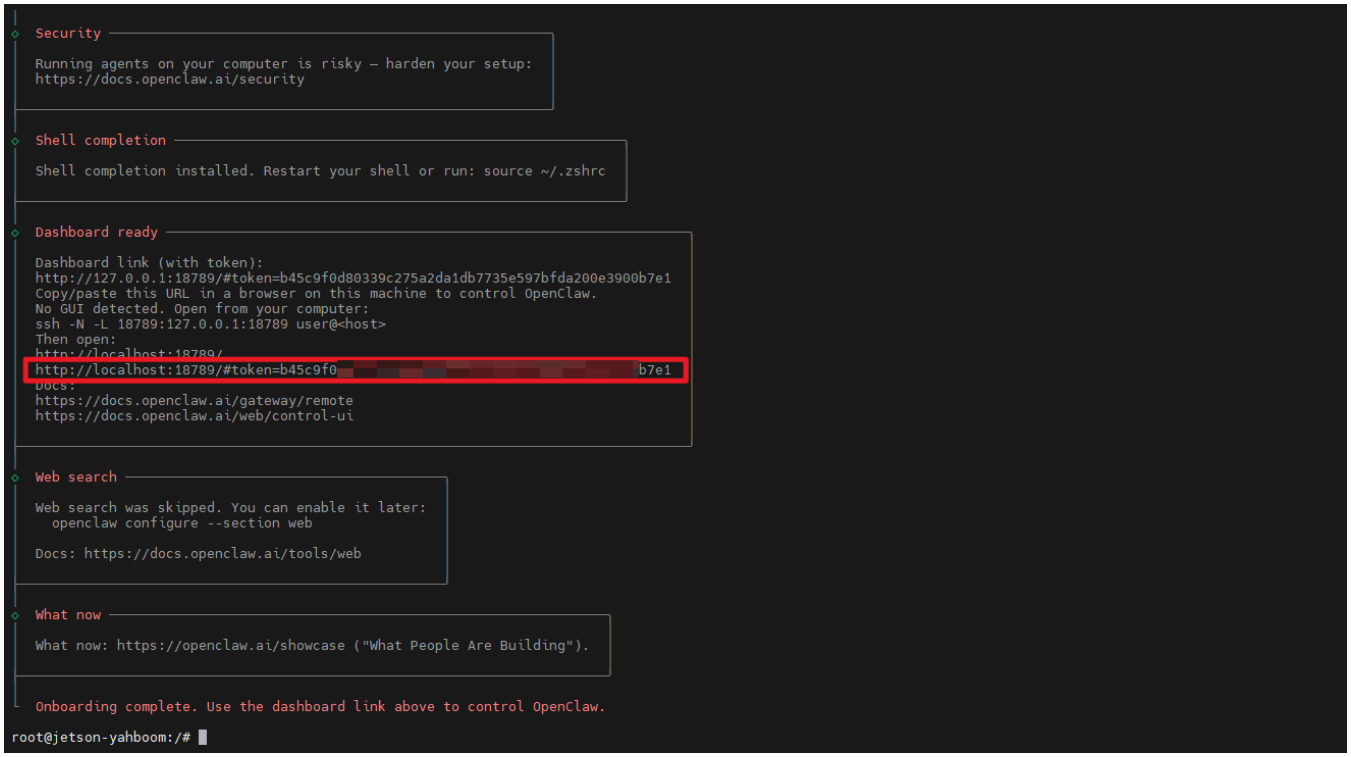


For the Hooks configuration, we press the Spacebar to select "Skip for now", and then press the Enter key to confirm



Wait for the installation to complete.

After the installation is complete, we copy this address for later use (in the subsequent tutorials the token changes because we configure the Gateway manually later on, so this address actually does not need to be recorded; it will change again later)



1.1.3 Opening the OpenClaw Control UI on a PC (Windows)

> Common issue note: please use a recent version of the Edge or Chrome browser; other browsers may report a 4008 error.

After OpenClaw version 3.8 is deployed in Docker on the Jetson Nano, accessing the OpenClaw Control UI (i.e. the web console) through a browser on the Jetson itself results in a 4008 access error, so we need to open a browser on another computer.

> If no other computer is available, we can use OpenClaw's TUI tool and operate it from the terminal in command line mode; see the TUI usage section later in this document for the detailed steps.

1. Modify the OpenClaw configuration file to allow other devices on the LAN to access OpenClaw

```
openclaw config
```

Here we make our selections the way shown in the figure below (use the Up/Down arrow keys to select and press Enter to confirm)

> Except for the Token, which may differ from the one in our figure, everything else must be the same. The key item is the Gateway bind mode option; we need to set it to LAN, so that other devices on our LAN can access our OpenClaw



Finally we select Continue to confirm and press Enter, and the configuration is complete!



After the configuration is complete, first start the OpenClaw Gateway. Open another terminal and enter the following in the terminal

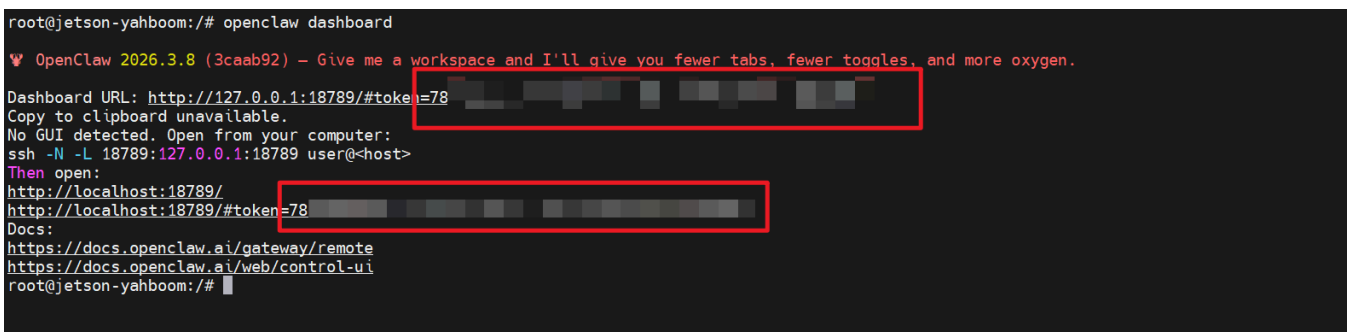
```
openclaw gateway
```

2. Get the access Token

The Token here is the one generated during the configuration we just did. If you did not write it down, you can use

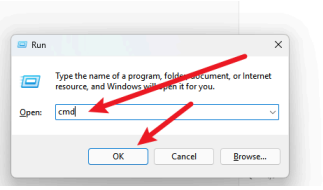
```
openclaw dashboard
```

This command displays the token once, or you can find the ~/.openclaw/openclaw.json file; your token is stored in this file



3. On a PC in the same LAN, configure the environment for connecting to the Jetson

Open the Cmd terminal on the computer (taking a Windows PC as an example, press Win+R to bring up the Run utility, type cmd, and then press Enter)

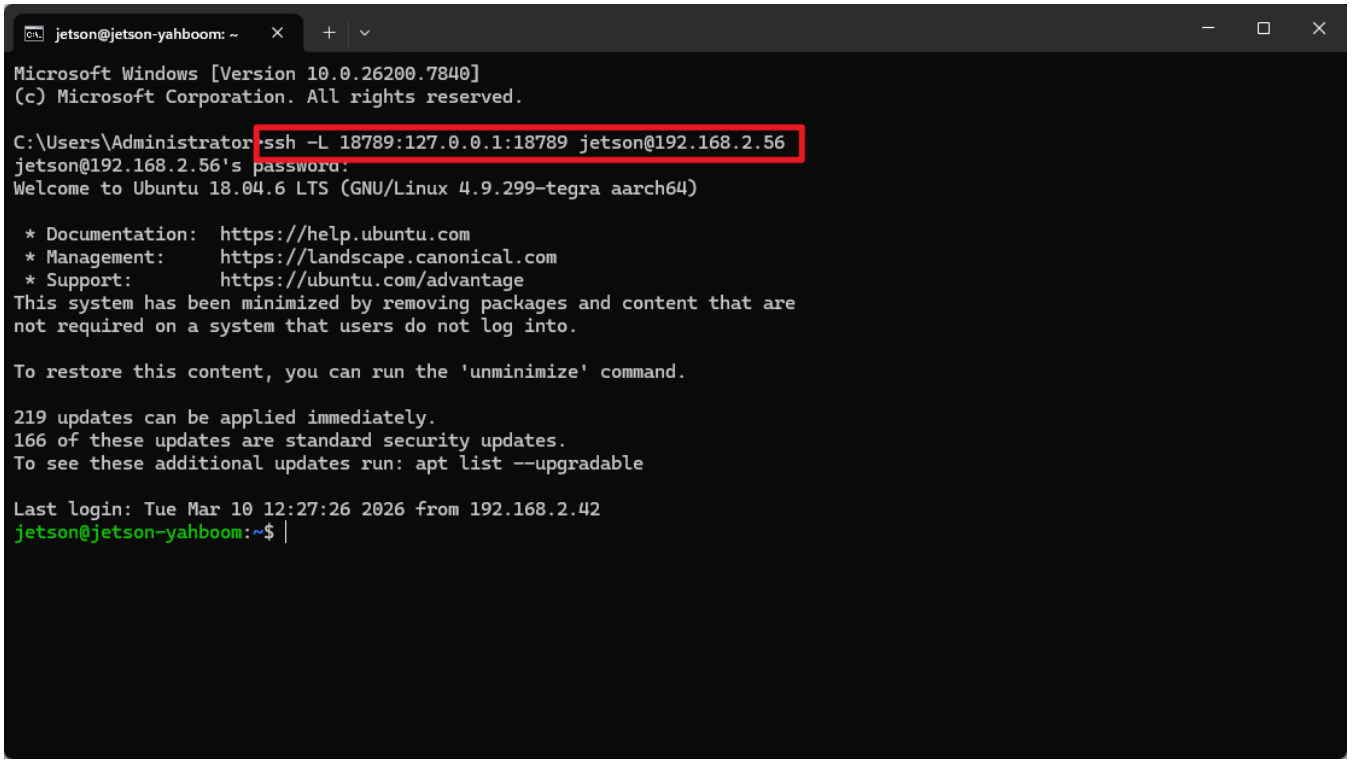


Enter the following in the console

```
ssh -L 18789:127.0.0.1:18789 jetson@<jetson-ip-address>
```

Note: here must be replaced with the actual IP address of your Jetson board

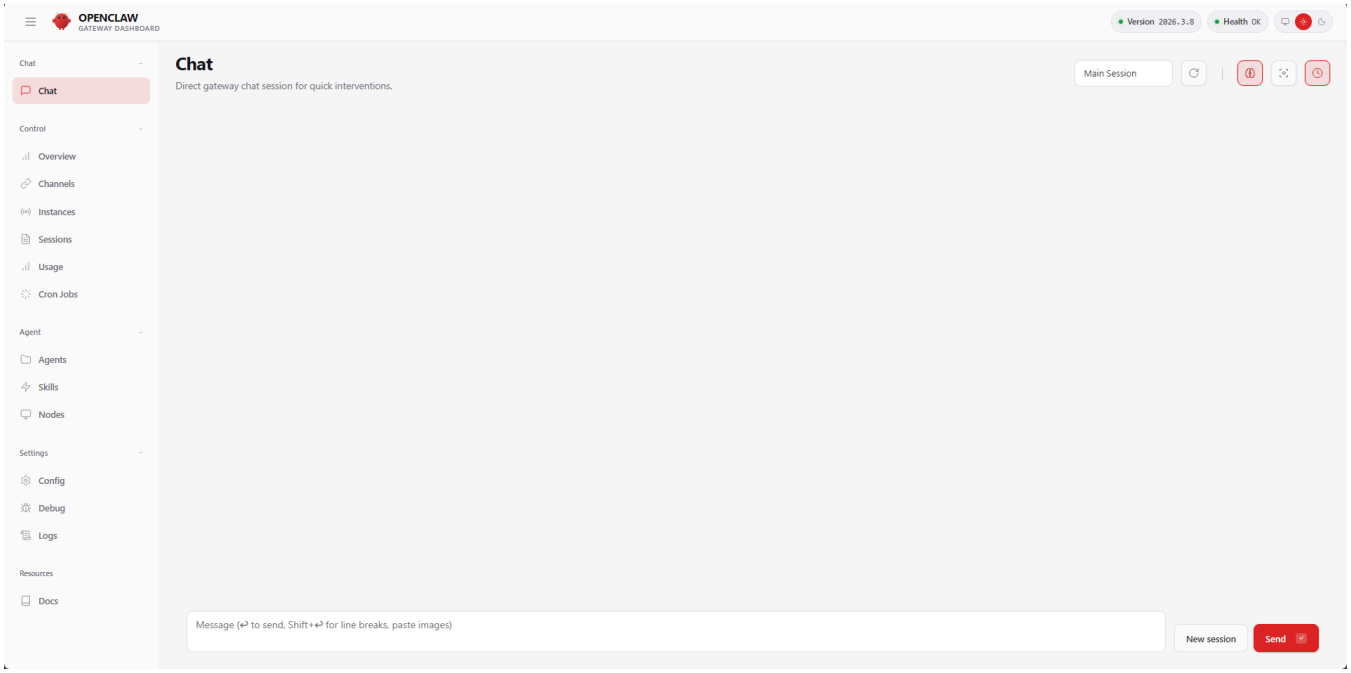
After pressing Enter you need to enter the password (the default is yahboom)



When this screen appears, it means you have successfully connected to the Jetson

4. Access it in the browser

Open the browser on the PC and enter the address consisting of the IP address and the token (the address output by openclaw dashboard just now, https://127.0.0.1:18789/#token=xxxxxxxxxxxx). When this interface appears, it means the deployment was successful!



1.2 Configuring the Model (Must Read)

1.2.1 Why a model must be configured

OpenClaw is an AI agent gateway; by itself it does not have the "intelligent" capabilities of a large model. We need to configure a large model to give OpenClaw the ability to think and understand.

1.2.2 How to configure a model

Here we use three representative models as examples, namely

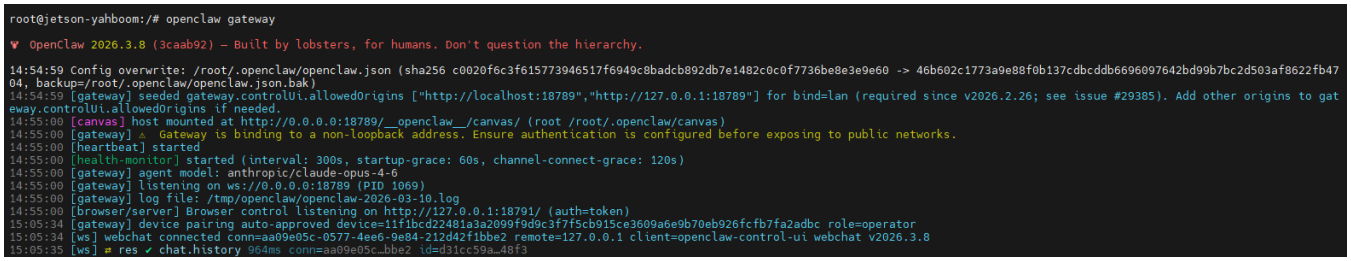
1. OpenRouter (an OpenClaw built-in provider, accessible both domestically and internationally)
2. Alibaba Bailian Large Model Platform (not an OpenClaw provider, needs manual configuration)
3. Ollama (a local large model)

Before starting the configuration, we need to start the OpenClaw Gateway first.

> If you did not exit the Gateway terminal after completing the deployment in the previous chapter, you can skip this step

Enter the following in the terminal

```
openclaw gateway
```



1. OpenRouter (an OpenClaw built-in provider)

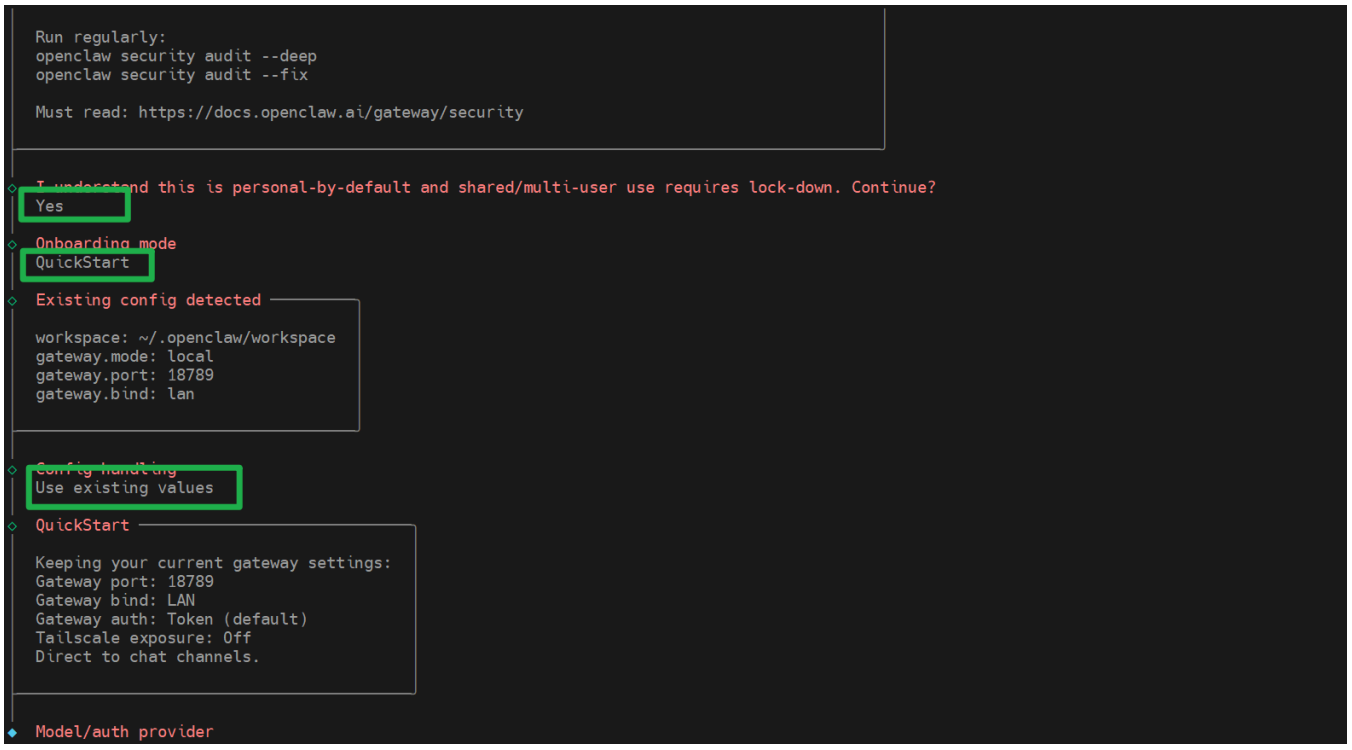
Here you need to have already applied for the OpenRouter API Key according to the previous tutorial (please refer to the previous tutorial [Online Large Models / 4. OpenRouter Large Model API Aggregation Platform])

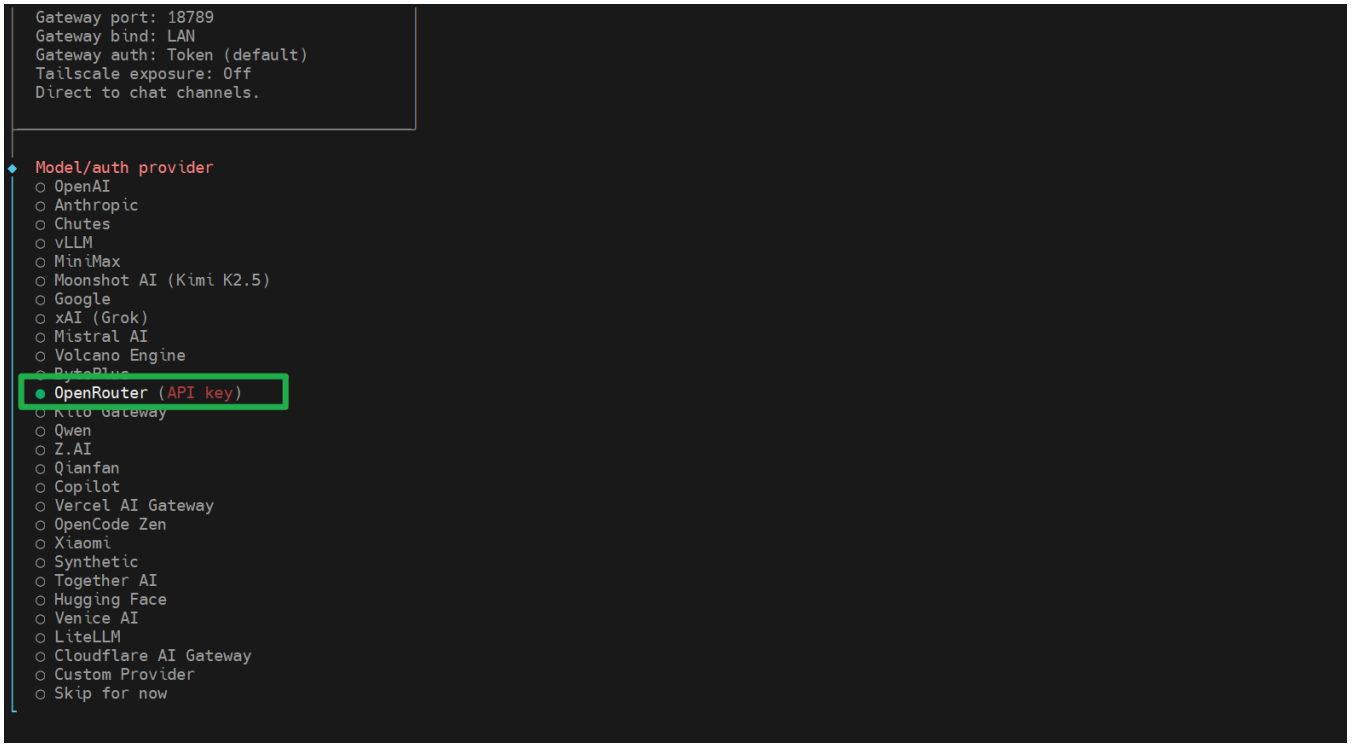
The OpenRouter API Key format looks like sk-or-v1-b3exxxxxx4xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxcddf81d9

We still use the initial onboard wizard to configure

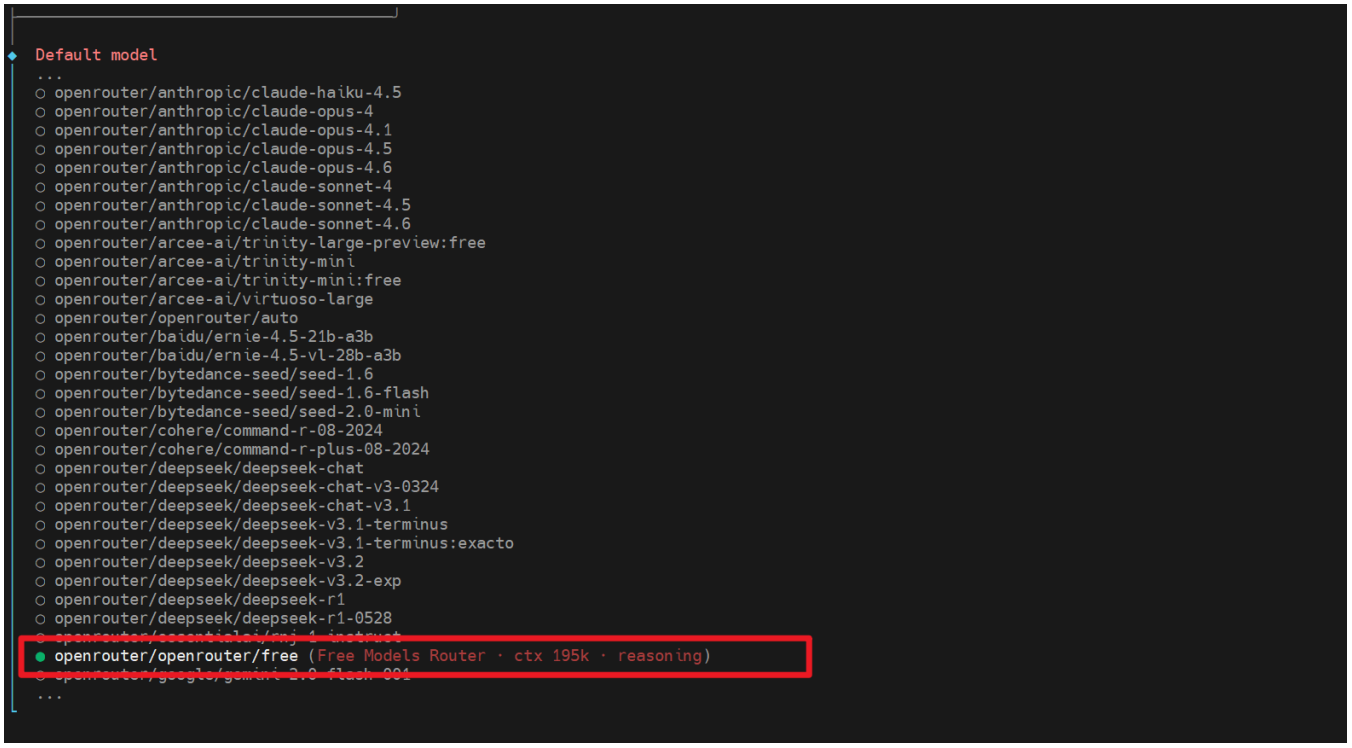
```
openclaw onboard
```

Please make your selections as shown in my figures



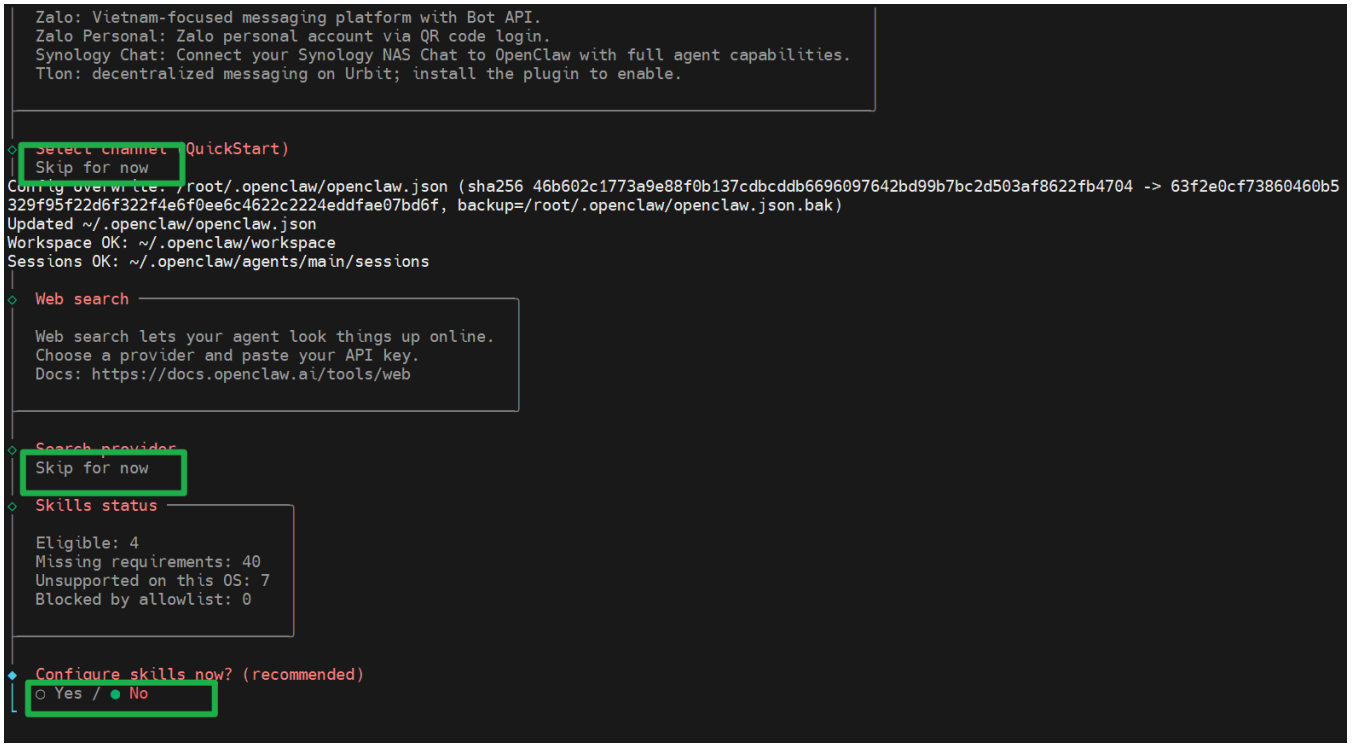


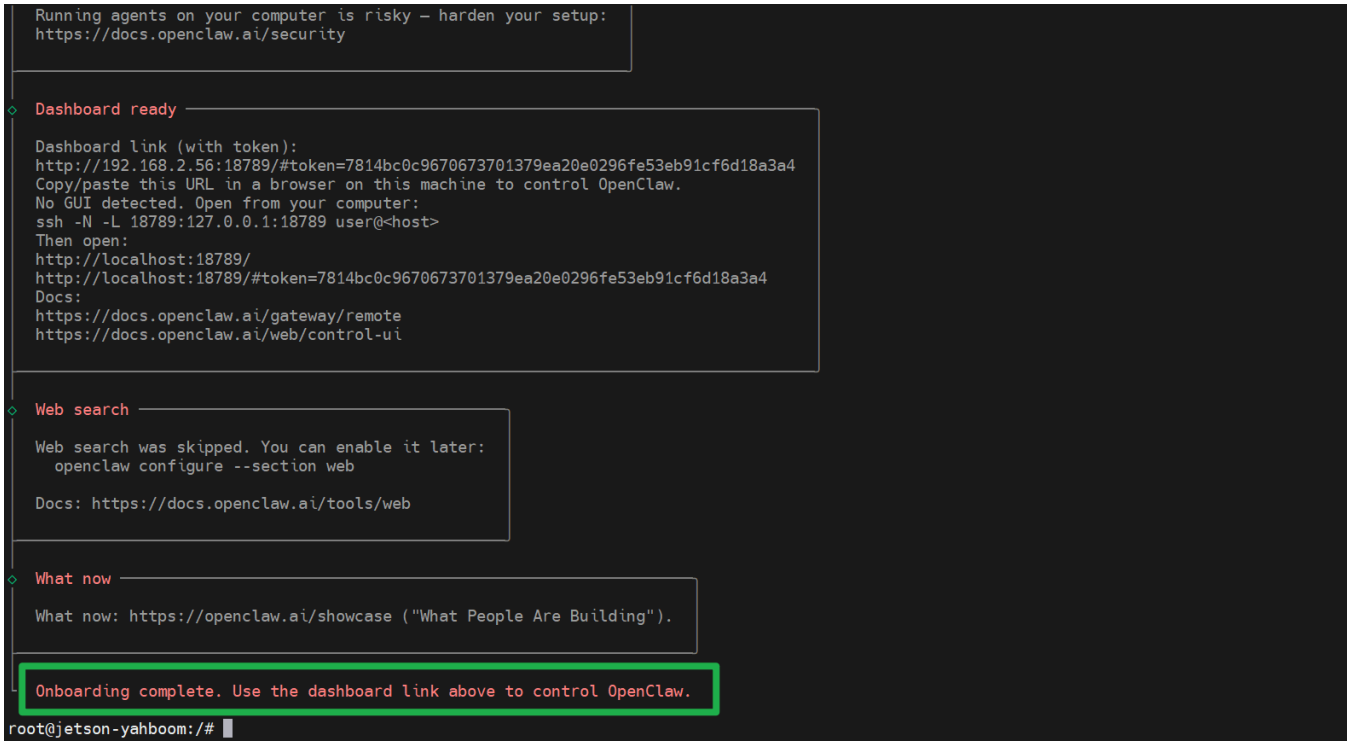
In the next step, we need to select a model. Due to OpenRouter's traffic control policy, we recommend that you keep a balance of $11 or more in your OpenRouter account; if the balance is insufficient, OpenRouter may throttle your requests (this policy may change in the future; please refer to the latest actual situation). Here we can choose a free model (the model name ends with free), or choose the free dedicated model openrouter/free on OpenRouter (this is a router that randomly picks a free model from those available on OpenRouter. This route intelligently filters out the models that support the functionality required by your request)



> At this point an option asking for the API Key may appear; paste the API Key in and then press Enter to go to the next step

After that we reach the other configuration items; we skip them for now and select Skip for all of them





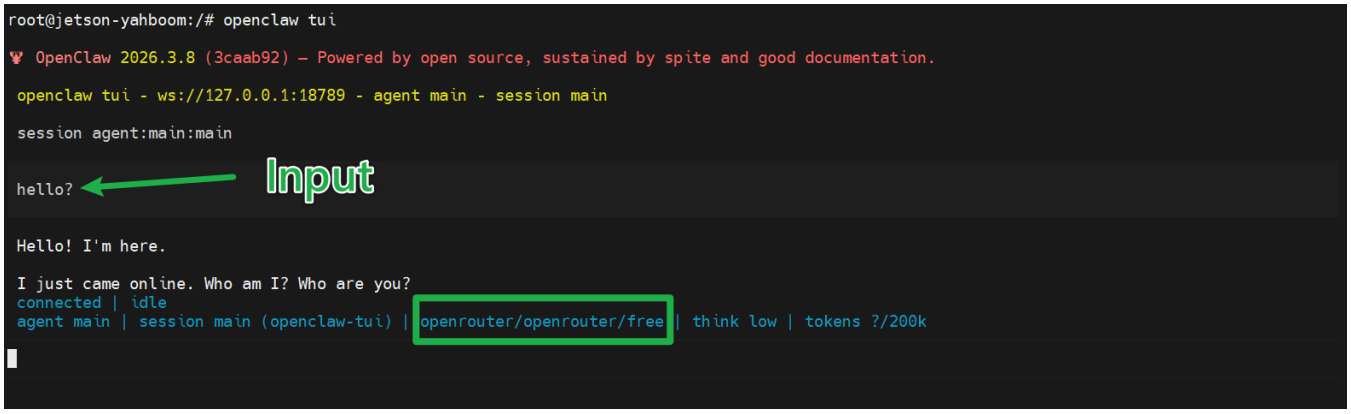
At this point, the configuration is complete

We can use the TUI feature to do a quick test

Enter the command

```
openclaw tui
```

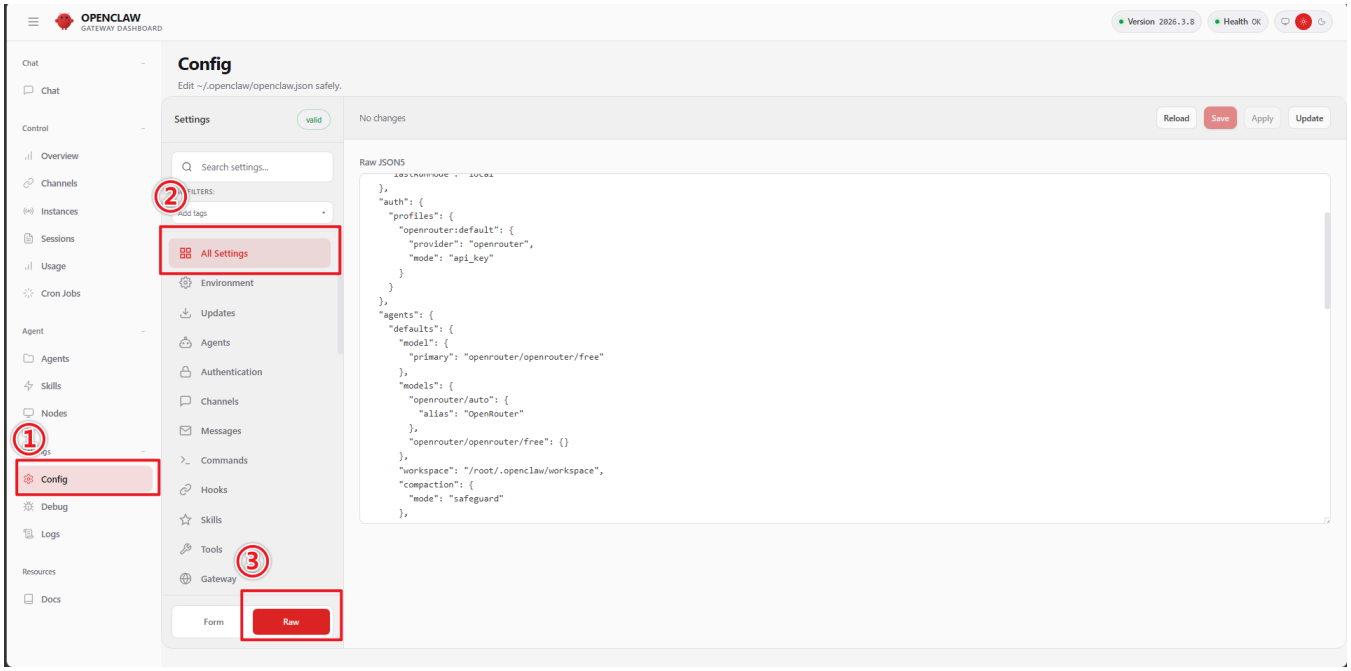
After entering it, simply type a sentence to greet openclaw (the Input section in the figure is what we typed, and the green box is the model we selected)



2. Bailian (an example of a third-party provider API)

Configuring a third-party API requires editing the OpenClaw configuration file (located at ~/.openclaw/openclaw.json); here I recommend editing it on the web page (the Openclaw Control UI) instead

Open the OpenClaw configuration page in the browser and click these three buttons in order



The current contents are as follows; what we need to pay attention to is the agents section


```
{
  "meta": {
    "lastTouchedVersion": "2026.3.8",
    "lastTouchedAt": "2026-03-10T07:56:18.292Z"
  },
  "wizard": {
    "lastRunAt": "2026-03-10T07:56:18.270Z",
    "lastRunVersion": "2026.3.8",
    "lastRunCommand": "onboard",
    "lastRunMode": "local"
  },
  "auth": {
    "profiles": {
      "openrouter:default": {
        "provider": "openrouter",
        "mode": "api_key"
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "openrouter/openrouter/free"
      },
      "models": {
        "openrouter/auto": {
          "alias": "OpenRouter"
        },
        "openrouter/openrouter/free": {}
      },
      "workspace": "/root/.openclaw/workspace",
      "compaction": {
```

```
      "mode": "safeguard"
    },
    "maxConcurrent": 4,
    "subagents": {
      "maxConcurrent": 8
    }
  }
},
"tools": {
  "profile": "coding"
},
"messages": {
  "ackReactionScope": "group-mentions"
},
"commands": {
  "native": "auto",
  "nativeSkills": "auto",
  "restart": true,
  "ownerDisplay": "raw"
},
"session": {
  "dmScope": "per-channel-peer"
},
"gateway": {
  "port": 18789,
  "mode": "local",
  "bind": "lan",
  "controlUi": {
    "allowedOrigins": [
      "http://localhost:18789",
      "http://127.0.0.1:18789"
    ]
  },
  "auth": {
    "mode": "token",
    "token": "__OPENCLAW_REDACTED__"
  },
  "tailscale": {
    "mode": "off",
    "resetOnExit": false
  },
  "nodes": {
    "denyCommands": [
      "camera.snap",
      "camera.clip",
      "screen.record",
      "contacts.add",
      "calendar.add",
      "reminders.add",
      "sms.send"
    ]
  }
}
```

```
}
```

We need to add a Models section (in the first level of the JSON, that is, this newly added "models" is at the same level as "agents")

Note: here you need to change DASHSCOPE_API_KEY to your real Bailian API Key, and change baseUrl to the OpenAI-compatible address matching your own account (see the previous chapter for the application method)

Here we use qwen3.5-plus and qwen3-coder-next as examples for models; if you want to support other models, please refer to the model list at https://help.aliyun.com/zh/model-studio/models


3. 添加百炼配置
复制以下配置信息，替换原 "agents": {...}, 部分，并将 DASHSCOPE_API_KEY 替换为您的百炼 API Key。

重要 本文档仅适用于按量付费模式，Coding Plan 需要使用专属 Base URL 和 API Key 接入，且仅支持特定的模型，详情请参考 OpenClaw Coding Plan 说明文档进行配置。
以下配置使用北京地域的 qwen3.5-plus 和 qwen3-coder-next 模型，请确保 Base URL 和 API Key 地域一致。
如需使用其他模型，请在 providers.bailian.models 中添加模型定义（reasoning 参数需设为 false），在 agents.defaults.models 中添加 "bailian/模型 ID": {} 条目。按量付费模式下各地域支持的模型，请参见模型列表。

```
"models": {
  "mode": "merge",
  "providers": {
    "bailian": {
      "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
      "apiKey": "DASHSCOPE_API_KEY",
      "api": "openai-completions",
      "models": [
        {
          "id": "qwen3.5-plus",
          "name": "qwen3.5-plus",
          "reasoning": false,
          "input": ["text", "image"],
          "contextWindow": 1000000,
          "maxTokens": 65536
        },
        {
          "id": "qwen3-coder-next",
          "name": "qwen3-coder-next",
          "reasoning": false,
          "input": ["text"],
          "contextWindow": 262144,
          "maxTokens": 65536
        }
      ]
    }
  }
},
```

After adding the "models", we also need to modify the "agents" part to set the Bailian large model as the default.

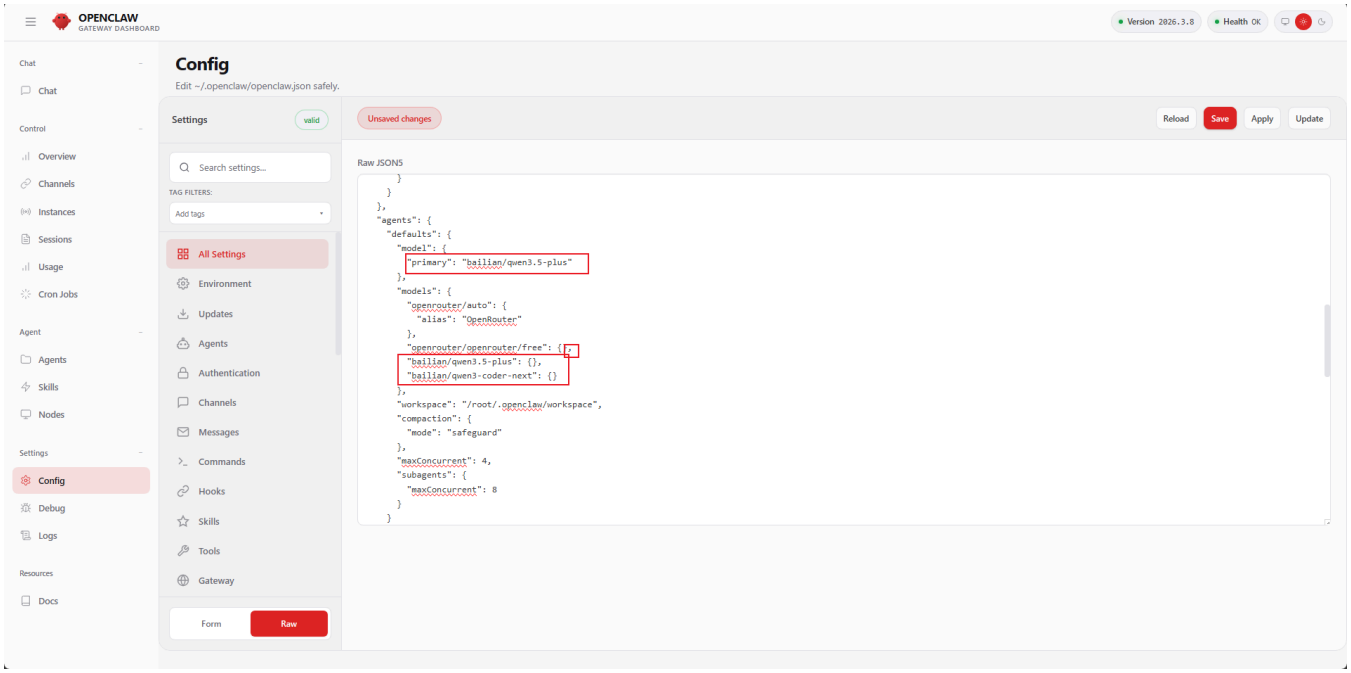
The agents before modification are as follows (by default it uses the OpenRouter we configured in the previous step)

```
"agents": {
```

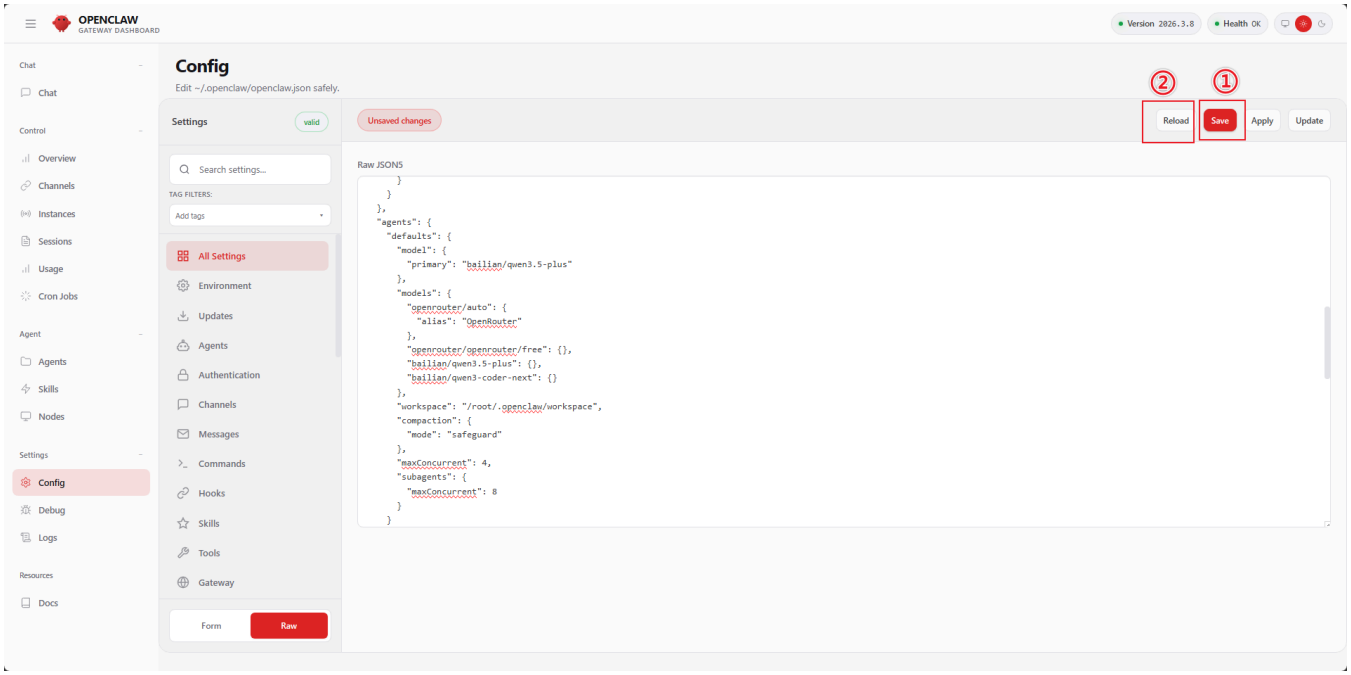
```
    "defaults": {
      "model": {
        "primary": "openrouter/openrouter/free"
      },
      "models": {
        "openrouter/auto": {
          "alias": "OpenRouter"
        },
        "openrouter/openrouter/free": {}
      },
      "workspace": "/root/.openclaw/workspace",
      "compaction": {
        "mode": "safeguard"
      },
      "maxConcurrent": 4,
      "subagents": {
        "maxConcurrent": 8
      }
    }
  },
```

The agents after modification are as follows

```
  "agents": {
    "defaults": {
      "model": {
        "primary": "bailian/qwen3.5-plus"
      },
      "models": {
        "openrouter/auto": {
          "alias": "OpenRouter"
        },
        "openrouter/openrouter/free": {},
        "bailian/qwen3.5-plus": {},
        "bailian/qwen3-coder-next": {}
      },
      "workspace": "/root/.openclaw/workspace",
      "compaction": {
        "mode": "safeguard"
      },
      "maxConcurrent": 4,
      "subagents": {
        "maxConcurrent": 8
      }
    }
  },
```

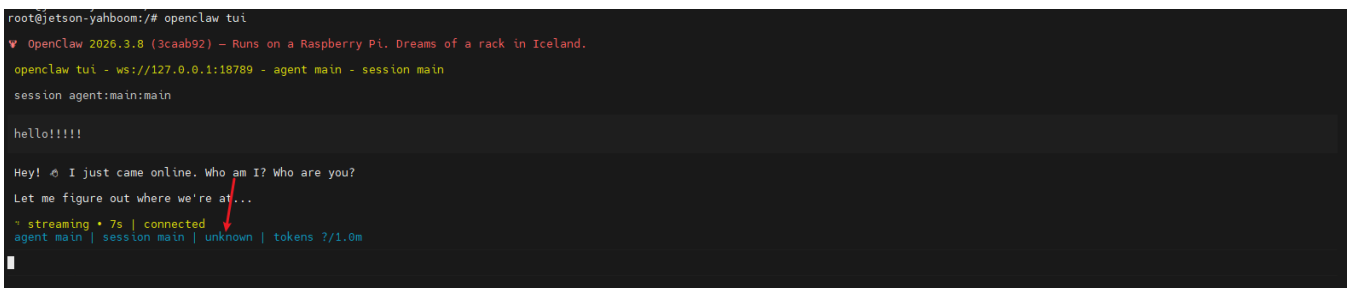


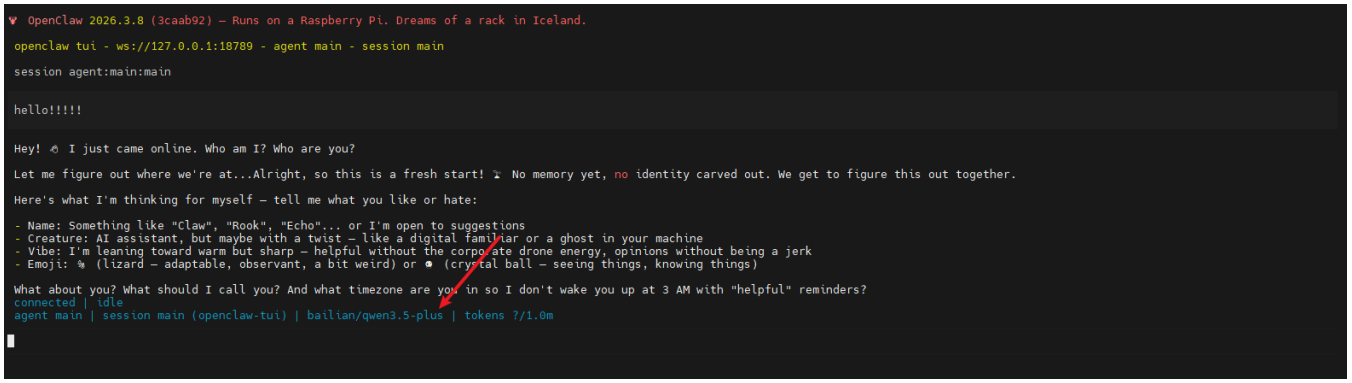
After making the changes, remember to save and reload



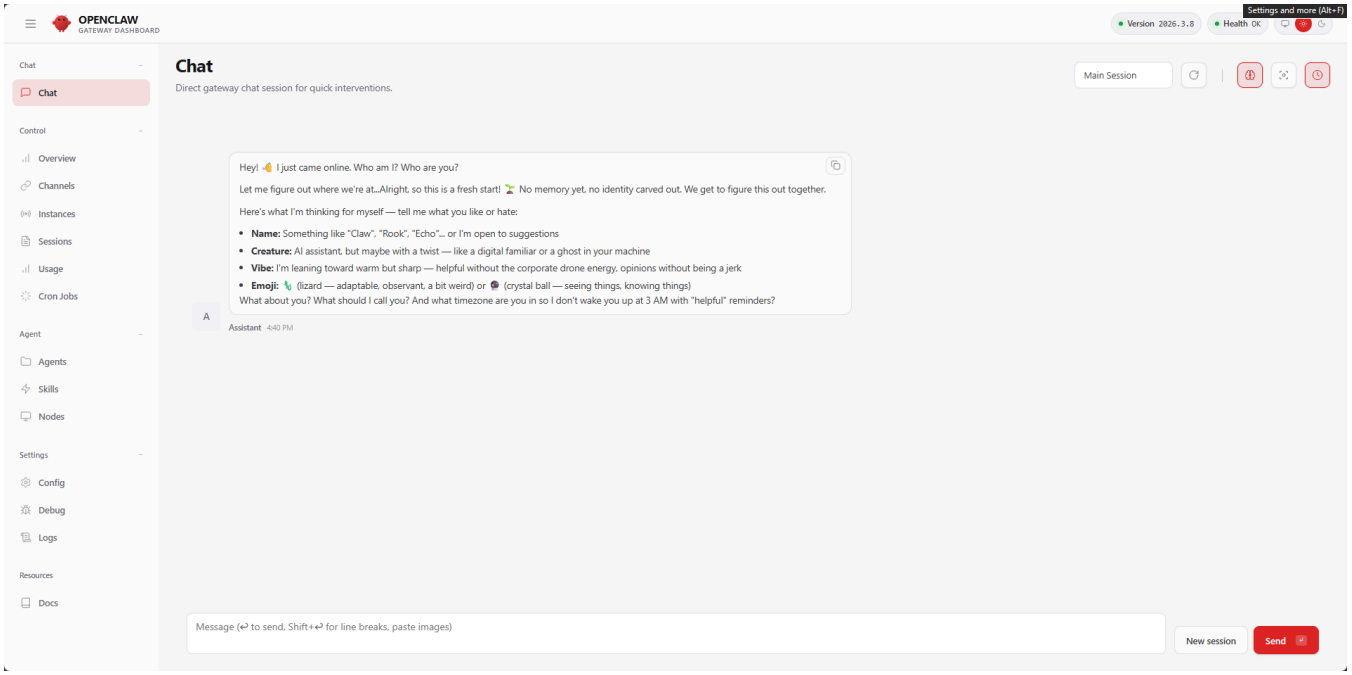
> Version 2026.3.8 supports hot reloading of the configuration file; after clicking Save, there is no need to restart the openclaw service. Generally speaking, restarting openclaw tui is enough. If restarting openclaw tui does not take effect, try restarting openclaw gateway again.

At the beginning it shows unknown; after waiting for a while the current model can be correctly identified





Similarly, besides the TUI interface, we can also chat and converse in Openclaw's Control UI (that is, the web front end)



We can actually observe that the conversation content in the TUI is the same as in our Control UI. This is because OpenClaw's conversations use a Session mechanism to separate different sessions; our TUI and the Control UI are actually in the same Session. You can try clicking the New session button in the bottom-right corner to start a new round of conversation